ESPRIT · Module MT-41 · Applications Web Distribuées

Mini-projet 2026-2027 · Validation intermédiaire (séance 4)

Refugia

Dossier technique

Application de gestion d'un refuge pour animaux en architecture microservices

Classe 4DAS_I13 · Groupe InSync 3

Youssef · Zeineb · Aziz · Emna · Lazhar · Kmar

Microservices : Animaux · Demandes d'adoption · Familles d'accueil · Suivi vétérinaire · Dons et parrainages · Visites et rendez-vous · Notifications

Novembre 2026

Sommaire

1. Description du sujet 3
1.1 Contexte et problématique 3
1.2 Objectifs et périmètre 3

2. Acteurs et fonctionnalités 4
2.1 Acteurs 4
2.2 Cas d'utilisation général 4
2.3 Fonctionnalités principales 4

3. Découpage en microservices 5

4. Données gérées par chaque service 6
4.1 MS1 · Animaux 6
4.2 MS2 · Demandes d'adoption 6
4.3 MS3 · Familles d'accueil 6
4.4 MS4 · Suivi vétérinaire 7
4.5 MS5 · Dons et parrainages 7
4.6 MS6 · Visites et rendez-vous 7
4.7 MS7 · Notifications (service commun) 7

5. Architecture globale 9
5.1 Diagramme d'architecture 9
5.2 Composants et ports 9
5.3 Routage de l'API Gateway 10
5.4 Sécurité 10
5.5 Communications entre services 10

6. Interactions entre les services 11
6.1 Appels synchrones 11
6.2 Catalogue des événements 11
6.3 Scénario de bout en bout : adopter un animal 12

7. Conception détaillée de chaque microservice 13
7.1 MS1 · Animaux 13
7.2 MS2 · Demandes d'adoption 16
7.3 MS3 · Familles d'accueil 19
7.4 MS4 · Suivi vétérinaire 22
7.5 MS5 · Dons et parrainages 25
7.6 MS6 · Visites et rendez-vous 28
7.7 MS7 · Notifications (service commun) 31

8. Technologies et conventions communes 34
8.1 Technologies choisies 34
8.2 Conventions communes 34

9. Répartition des tâches 35

10. Organisation du dépôt Git et gestion de projet 35
10.1 Dépôt 35
10.2 Outil de gestion de projet 35

11. Planning prévisionnel et risques 35
11.1 Planning 35
11.2 Principaux risques 36

1. Description du sujet

1.1 Contexte et problématique

Un refuge pour animaux gère en même temps l'accueil et le suivi des animaux, les demandes d'adoption, les familles d'accueil, les soins vétérinaires, les dons et les visites du public. Ces informations sont souvent dispersées entre des fiches papier, des tableurs et des messages : statuts qui ne sont pas à jour, demandes perdues, soins oubliés, donateurs sans reçu.

Problématique : comment centraliser la gestion d'un refuge dans une application où chaque domaine (animaux, adoptions, accueil, soins, dons, visites) est géré de façon autonome, tout en échangeant avec les autres sans dépendre d'eux ?

Refugia y répond par une application web en **architecture microservices** : chaque domaine est un service Spring Boot indépendant avec sa propre base de données ; les services communiquent par des appels REST synchrones et par des événements asynchrones.

1.2 Objectifs et périmètre

- **Pour le public** : consulter les animaux à adopter, réserver une visite, déposer une demande d'adoption, faire un don ou parrainer un animal.
- **Pour le personnel** : suivre les animaux, traiter les demandes, gérer les familles d'accueil, le suivi vétérinaire, les créneaux de visite et les événements.
- **Pour l'équipe** : sept microservices (six individuels et un service commun), une API Gateway, Eureka, Keycloak, RabbitMQ et une application React, conformément à la stack du module MT-41.
- **Hors périmètre** : paiement en ligne réel (le don enregistre seulement un mode de paiement), comptabilité, stocks, ressources humaines, application mobile native.

2. Acteurs et fonctionnalités

2.1 Acteurs

Trois rôles sont gérés dans Keycloak. Le vétérinaire est un membre du personnel qui appartient en plus au groupe Keycloak « veterinaire » : ce groupe lui donne le droit d'écrire dans le suivi vétérinaire (MS4), que les autres membres du personnel ne font que consulter. Un visiteur sans compte peut consulter le catalogue des animaux, les créneaux de visite et les événements.

Rôle	Qui	Ce qu'il fait
`CLIENT`	Visiteur, adoptant, donateur, parrain, famille d'accueil	Consulte les animaux, réserve une visite, s'inscrit à un événement, dépose et suit une demande d'adoption, fait un don, parraine un animal, consulte ses notifications
`PERSONNEL`	Employés du refuge et vétérinaire	Gère les fiches animaux, traite les demandes d'adoption, gère les familles d'accueil et les placements, organise créneaux et événements, consulte les dons ; le vétérinaire gère en plus les dossiers médicaux
`ADMIN`	Responsable du refuge	A tous les droits du personnel, plus : suppression d'une fiche saisie par erreur, statistiques, annulation d'un événement ou d'un don, renvoi d'une notification, gestion des comptes et des rôles

2.2 Cas d'utilisation général

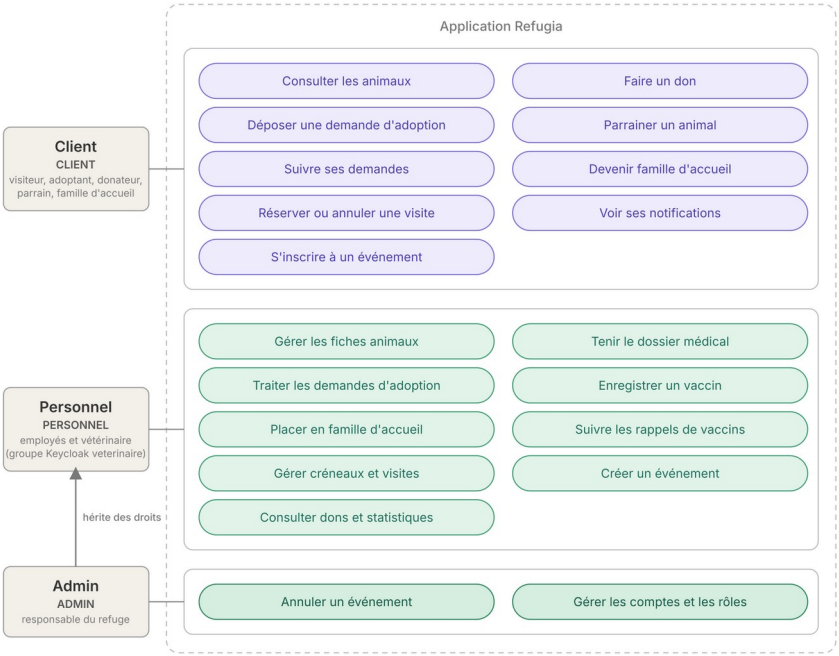


Figure 1 : Acteurs et actions de l'application Refugia (vue générale)

2.3 Fonctionnalités principales

Domaine	Fonctionnalités	Service
Animaux	Fiches, recherche et filtres, statut avec transitions contrôlées, photos, historique, statistiques	MS1
Adoption	Profil du candidat, demande d'adoption, score de compatibilité, acceptation ou refus, finalisation	MS2
Familles d'accueil	Familles et capacité, placement d'un animal, fin ou annulation d'un placement	MS3
Suivi vétérinaire	Dossier médical, consultations, vaccinations avec rappels, traitements	MS4

Domaine	Fonctionnalités	Service
Dons et parrainages	Dons avec reçu, parrainages mensuels, donateurs, statistiques de dons	MS5
Visites et événements	Créneaux, réservation et annulation de visites, événements, inscriptions et liste d'attente	MS6
Notifications	E-mails et notifications internes déclenchés par les événements des autres services	MS7

3. Découpage en microservices

Le découpage suit les domaines métier : un service est responsable d'un seul domaine et de ses données. MS1 est le service de référence sur les animaux ; les autres ne connaissent un animal que par son identifiant.

Service	Responsable	Port	Responsabilité	Base
MS1 Animaux	Youssef	8010	Fiches, statuts, photos, historique	MySQL
MS2 Demandes d'adoption	Zeineb	8020	Profil candidat, demandes, score, finalisation	MySQL
MS3 Familles d'accueil	Aziz	8030	Familles, capacité, placements	MySQL
MS4 Suivi vétérinaire	Emna	8040	Dossier médical, consultations, vaccins, traitements	MySQL
MS5 Dons et parrainages	Lazhar	8050	Donateurs, dons, reçus, parrainages, statistiques	MySQL
MS6 Visites et rendez-vous	Kmar	8060	Créneaux, visites, événements, inscriptions	MySQL
MS7 Notifications (commun)	Toute l'équipe	8070	Notifications e-mail et internes	MongoDB

Service	Appelle (synchrone)	Publie	Écoute
MS1	–	AnimalCréé, StatutAnimalModifié	–
MS2	MS1	4 événements d'adoption	–
MS3	MS1	PlacementCréé, PlacementTerminé	AdoptionFinalisée
MS4	MS1	RappelVaccination, ConsultationEnregistrée	–
MS5	MS1	DonValidé, ParrainageCréé, ParrainageTerminé	AdoptionFinalisée
MS6	MS1	4 événements de visites	StatutAnimalModifié
MS7	–	–	Tous les événements (17)

Pourquoi ce découpage. Chaque service peut être compris, testé et déployé seul. La dépendance est à sens unique : tous dépendent de MS1 pour connaître un animal, MS1 ne dépend d'aucun autre. Le service commun MS7 n'appelle personne : il réagit aux événements. Il est développé par toute l'équipe (chaque membre y ajoute l'écouteur et le message de son propre service) et présenté par un seul membre.

4. Données gérées par chaque service

Chaque service possède sa propre base et ne lit jamais celle d'un autre. Un service qui parle d'un animal garde seulement son identifiant `animalId` (sans clé étrangère) et interroge MS1 pour le reste. Quand une information stable est copiée (par exemple `nomAnimal` dans MS6), elle l'est une fois, au moment de l'action.

4.1 MS1 · Animaux

Données (base MySQL {{animal_db}})

Entité	Attributs principaux
Animal	id, nom, espece, race, sexe, dateNaissance, taille, description, niveauEnergie, ententeEnfants, ententeAutresAnimaux, statut, dateArrivee
Photo	id, animalId, url, principale, dateAjout
HistoriqueStatut	id, animalId, ancienStatut, nouveauStatut, dateChangement, origine, motif

Énumération	Valeurs
StatutAnimal	DISPONIBLE, RESERVE, EN_ACCUEIL, EN_SOINS, ADOPTE
Espece	CHIEN, CHAT, LAPIN, OISEAU, AUTRE
Sexe / Taille	MALE, FEMELLE / PETIT, MOYEN, GRAND

`niveauEnergie` (de 1 à 5), `ententeEnfants` et `ententeAutresAnimaux` servent au score de compatibilité calculé par MS2. `origine` indique qui a déclenché un changement : l'identifiant d'un membre du personnel ou le nom du service appelant.

4.2 MS2 · Demandes d'adoption

Données (base MySQL)

Entité	Attributs principaux
DemandeAdoption	id, animalId, clientId, statut, motifRefus, dateCreation, dateDecision
ProfilCandidat	id, clientId, typeLogement (APPARTEMENT, MAISON), jardin, nbPersonnes, enfants, autresAnimaux, experienceAnimaux, disponibilites
ScoreCompatibilite	id, scoreTotal, dateCalcul

Statuts d'une demande : `EN_ATTENTE`, `EN_ETUDE`, `ACCEPTEE`, `REFUSEE`, `ANNULEE`, `FINALISEE`. `animalId` et `clientId` sont de simples références : MS2 ne stocke aucune donnée de MS1.

4.3 MS3 · Familles d'accueil

Données (base MySQL)

Entité	Attributs principaux
FamilleAccueil	id, nom, telephone, email, adresse, capacite, especesAcceptees, statut (ACTIVE, INACTIVE)
Placement	id, animalId, familleId, dateDebut, dateFinPrevue, dateFinReelle, notes, motifFin, statut (EN_COURS, TERMINE, ANNULE)

4.4 MS4 · Suivi vétérinaire

Données (base MySQL)

Entité	Attributs principaux
DossierMedical	id, animalId (unique), numeroDossier, dateOuverture, notesGenerales
Consultation	id, dossierId, dateConsultation, motif, diagnostic, poids, temperature, remarques
Vaccination	id, dossierId, vaccin, dateVaccination, dateRappel, rappelPublie, statut (A_JOUR, A_RAPPELER, EN_RETARD)
Traitement	id, dossierId, medicament, posologie, dateDebut, dateFin, statut (EN_COURS, TERMINE, SUSPENDU)

4.5 MS5 · Dons et parrainages

Données (base MySQL)

Table	Colonnes principales
donateur	id, nom, email (unique), telephone, date_inscription
don	id, donateur_id (FK), animal_id, montant, mode_paiement, statut, date_don, message
parrainage	id, donateur_id (FK), animal_id, montant_mensuel, date_debut, date_fin, statut
recu	id, don_id (FK, unique), numero (unique), date_emission, montant

`animal_id` n'a pas de clé étrangère : la table des animaux est dans la base de MS1. Énumérations : ModePaiement (CARTE, VIREMENT, ESPECES), StatutDon (EN_ATTENTE, VALIDE, ANNULE), StatutParrainage (ACTIF, SUSPENDU, TERMINE).

4.6 MS6 · Visites et rendez-vous

Données (base MySQL)

Entité	Attributs principaux
Creneau	id, date, heureDebut, heureFin, capacite, placesRestantes
Visite	id, visiteurId, animalId, nomAnimal, espece, creneauId, statut (CONFIRMEE, ANNULEE, EFFECTUEE), motifAnnulation, dateCreation
Evenement	id, titre, type (JOURNEE_ADOPTION, ATELIER), description, date, lieu, capacite, placesRestantes, statut (PLANIFIE, ANNULE)
Inscription	id, evenementId, participantId, statut (INSCRIT, EN_ATTENTE, ANNULEE), dateInscription

`nomAnimal` et `espece` sont copiés depuis MS1 au moment de la réservation ; ils ne sont pas mis à jour ensuite.

4.7 MS7 · Notifications (service commun)

Données (collection MongoDB « notifications »)

Champ	Description
id	Identifiant du document
eventId	Identifiant de l'événement reçu (unique, pour l'idempotence)
typeEvenement	Nom de l'événement (DonValidé, VisiteCréée, etc.)

Champ	Description
destinataireId / destinataireEmail / role	Le destinataire : un utilisateur (identifiant Keycloak et e-mail) ou un rôle (PERSONNEL)
canal	EMAIL ou INTERNE
titre, message	Contenu construit à partir du gabarit du type d'événement
statut	EN_ATTENTE, ENVOYEE, ECHEC ou LUE
dateCreation, dateEnvoi	Dates de création et d'envoi

5. Architecture globale

5.1 Diagramme d'architecture

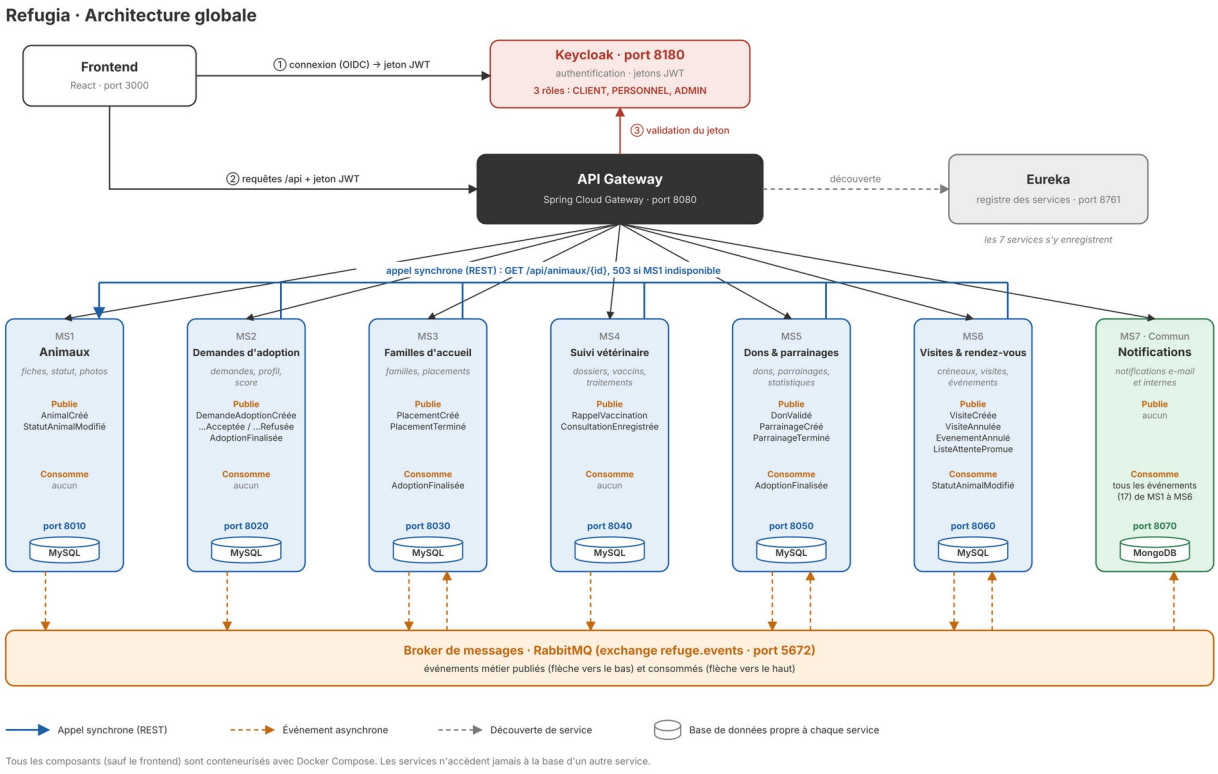


Figure 2 : Architecture globale de Refugia

5.2 Composants et ports

Composant	Rôle	Technologie	Port
Frontend	Application web pour les trois rôles ; appelle uniquement la Gateway	React	3000
API Gateway	Point d'entrée unique : routage, validation du jeton JWT, routes publiques en lecture	Spring Cloud Gateway	8080
Eureka	Registre : les services s'y enregistrent et se trouvent par leur nom	Eureka	8761
Keycloak	Authentification OIDC, jetons JWT, comptes, rôles, groupe « veterinaire »	Keycloak	8180
MS1 à MS6	Microservices métier, une base MySQL chacun	Spring Boot	8010 à 8060
MS7 Notifications	Service commun, base MongoDB	Spring Boot	8070
RabbitMQ	Événements asynchrones, exchange `refuge.events` (topic)	RabbitMQ	5672 (interface 15672)
Bases de données	Une base par service	MySQL / MongoDB	3306 / 27017

Chaque service définit son port dans `application.properties` (`server.port`), son nom Eureka (`spring.application.name`) et sa base. Les services MS1 à MS6 utilisent des schémas MySQL séparés.

5.3 Routage de l'API Gateway

Chemin exposé	Service	Sans compte
`/api/animaux/**`	MS1 Animaux (8010)	GET catalogue et détail
`/api/demandes/**`, `/api/profils/**`	MS2 (8020)	Non
`/api/familles/**`, `/api/placements/**`	MS3 (8030)	Non
`/api/vet/**`	MS4 (8040)	Non
`/api/donateurs/**`, `/api/dons/**`, `/api/parrainages/**`, `/api/stats/**`	MS5 (8050)	Non
`/api/visites/**`, `/api/evenements/**`	MS6 (8060)	GET créneaux et événements
`/api/notifications/**`	MS7 (8070)	Non

> `GET /api/animaux/{animalId}/parrains` appartient à MS5 mais commence comme les routes de MS1 : la Gateway déclare cette route plus précise en premier.

5.4 Sécurité

- **Authentification** : connexion à Keycloak (OIDC avec PKCE depuis React) ; l'utilisateur reçoit un jeton JWT avec son identifiant et ses rôles.
- **Gateway** : vérifie la signature et la durée du jeton sur toute route non publique (401 sinon).
- **Chaque service** : revérifie le jeton et applique ses règles par rôle (403 si insuffisant), donc une requête qui contourne la Gateway reste refusée.
- **Appels entre services** : le service appelant transmet le jeton de l'utilisateur d'origine.
- **Rôles** : ADMIN est un rôle composite qui contient PERSONNEL.

Service	Sans compte	CLIENT	PERSONNEL	ADMIN en plus
MS1 Animaux	Consulte le catalogue	Consulte	Crée, modifie, change le statut, photos	Supprime, statistiques
MS2 Demandes	–	Son profil et ses demandes	Traite, accepte, refuse, finalise	Supervise
MS3 Familles	–	–	Familles et placements	Mêmes droits
MS4 Vétérinaire	–	–	Consulte ; groupe « veterinaire » écrit	Consulte et écrit
MS5 Dons	–	Dons et parrainages (les siens)	Consulte dons et statistiques	Annule un don
MS6 Visites	Créneaux, événements	Réserve, s'inscrit	Gère créneaux et événements	Annule un événement
MS7 Notifications	–	Les siennes	Les siennes et celles du personnel	Liste complète, renvoi

5.5 Communications entre services

	Synchrone	Asynchrone
Technologie	REST avec OpenFeign, via Eureka	RabbitMQ (exchange topic `refuge.events`)
Usage	L'appelant a besoin de la réponse tout de suite (existence ou statut d'un animal)	L'émetteur informe d'un fait sans attendre de réponse
Si l'autre service est arrêté	503 et rien n'est enregistré (délai 2 s, Resilience4j)	Le message attend dans la file et est traité au redémarrage

6. Interactions entre les services

6.1 Appels synchrones

Tous les appels synchrones vont vers MS1, le service de référence des animaux, avec le jeton de l'utilisateur et un délai maximal de 2 secondes (puis 503).

Appelant	Appel vers MS1	Raison	Réponses
MS2	`GET /api/animaux/{id}`	Vérifier que l'animal existe et est disponible ; lire ses caractéristiques	404, 409, 503
MS2	`PATCH /api/animaux/{id}/statut`	Passer l'animal à ADOPTE à la finalisation	404, 409, 503
MS3	`GET /api/animaux/{id}`	Vérifier que l'animal est DISPONIBLE avant un placement	404, 409, 503
MS3	`PATCH /api/animaux/{id}/statut`	EN_ACCUEIL au placement, DISPONIBLE au retour	404, 409, 503
MS4	`GET /api/animaux/{id}`	Vérifier l'animal avant d'ouvrir un dossier	404, 503
MS5	`GET /api/animaux/{id}`	Vérifier l'animal avant un don ou un parrainage	404, 503
MS6	`GET /api/animaux/{id}`	Vérifier que l'animal est DISPONIBLE avant une visite	404, 409, 503

6.2 Catalogue des événements

Dix-sept événements circulent sur l'exchange `refuge.events`. Format commun : `eventId`, `eventType`, `occurredAt`, `data` ; clé de routage en minuscules ; une file par consommateur nommée `<service>.<clé>` ; un événement déjà traité (même `eventId`) est ignoré.

Événement	Clé de routage	Publié par	Consommé par	Contenu de data
AnimalCréé	animal.cree	MS1	MS7	animalId, nom, espece, statut
StatutAnimalModifié	animal.statut.modifie	MS1	MS6, MS7	animalId, ancienStatut, nouveauStatut, origine
DemandeAdoptionCréée	demande.creee	MS2	MS7	demandeId, clientId, animalId
DemandeAdoptionAcceptée	demande.acceptee	MS2	MS7	demandeId, clientId, animalId
DemandeAdoptionRefusée	demande.refusee	MS2	MS7	demandeId, clientId, animalId, motifRefus
AdoptionFinalisée	adoption.finalisee	MS2	MS3, MS5, MS7	demandeId, clientId, animalId
PlacementCréé	placement.cree	MS3	MS7	placementId, animalId, familleId, familleEmail, dateDebut, dateFinPrevue
PlacementTerminé	placement.termine	MS3	MS7	placementId, animalId, familleId, familleEmail, motif
RappelVaccination	vaccination.rappel	MS4	MS7	animalId, vaccin, dateRappel
ConsultationEnregistrée	consultation.enregistree	MS4	MS7	animalId, consultationId, date
DonValidé	don.valide	MS5	MS7	donId, donateurId, donateurEmail, animalId, montant, numeroRecu
ParrainageCréé	parrainage.cree	MS5	MS7	parrainageId, donateurId, donateurEmail, animalId, montantMensuel
ParrainageTerminé	parrainage.termine	MS5	MS7	parrainageId, donateurId, donateurEmail, animalId, motif
VisiteCréée	visite.creee	MS6	MS7	visiteId, visiteurId, animalId, date
VisiteAnnulée	visite.annulee	MS6	MS7	visiteId, visiteurId, animalId, motif

Événement	Clé de routage	Publié par	Consommé par	Contenu de data
EvenementAnnulé	evenement.annule	MS6	MS7	evenementId, titre, date, participantIds
ListeAttentePromue	liste-attente.promue	MS6	MS7	evenementId, titre, participantId

6.3 Scénario de bout en bout : adopter un animal

1. Le client réserve une visite (`POST /api/visites`, MS6) : MS6 appelle MS1 pour vérifier que l'animal est DISPONIBLE, puis publie `VisiteCréée` ; MS7 envoie la confirmation.
2. Le client dépose une demande (`POST /api/demandes`, MS2) : MS2 appelle MS1 et publie `DemandeAdoptionCréée`.
3. Le personnel accepte la demande ; MS2 publie `DemandeAdoptionAcceptée` ; MS7 prévient le client.
4. Le personnel finalise : MS2 appelle MS1 (`PATCH /api/animaux/17/statut`, ADOPTE) ; MS1 publie `StatutAnimalModifié`.
5. MS6 reçoit cet événement et annule les visites à venir de l'animal.
6. MS2 publie `AdoptionFinalisée` : MS3 termine le placement éventuel, MS5 termine les parrainages actifs, MS7 envoie les e-mails.

Aucun service ne lit la base d'un autre : MS1 reste la seule source de vérité sur l'animal.

7. Conception détaillée de chaque microservice

Pour chaque service : diagramme de classes, un exemple de communication synchrone et un exemple de communication asynchrone, puis le contrat d'API (endpoints, rôles, codes de réponse, événements). Les données de chaque service sont décrites en section 4.

7.1 MS1 · Animaux

Rôle

MS1 gère la fiche de chaque animal du refuge et son statut. C'est le service de référence du projet : MS2 à MS6 ne connaissent un animal que par son identifiant (`animalId`) et lisent MS1 pour le reste. MS1 n'appelle aucun autre service en synchrone.

Transitions de statut

Statut actuel	Statuts suivants autorisés
DISPONIBLE	RESERVE, EN_ACCUEIL, EN_SOINS, ADOPTE
RESERVE	DISPONIBLE, ADOPTE
EN_ACCUEIL	DISPONIBLE, RESERVE, EN_SOINS, ADOPTE
EN_SOINS	DISPONIBLE, EN_ACCUEIL
ADOPTE	Aucun (état final)

Diagramme de classes

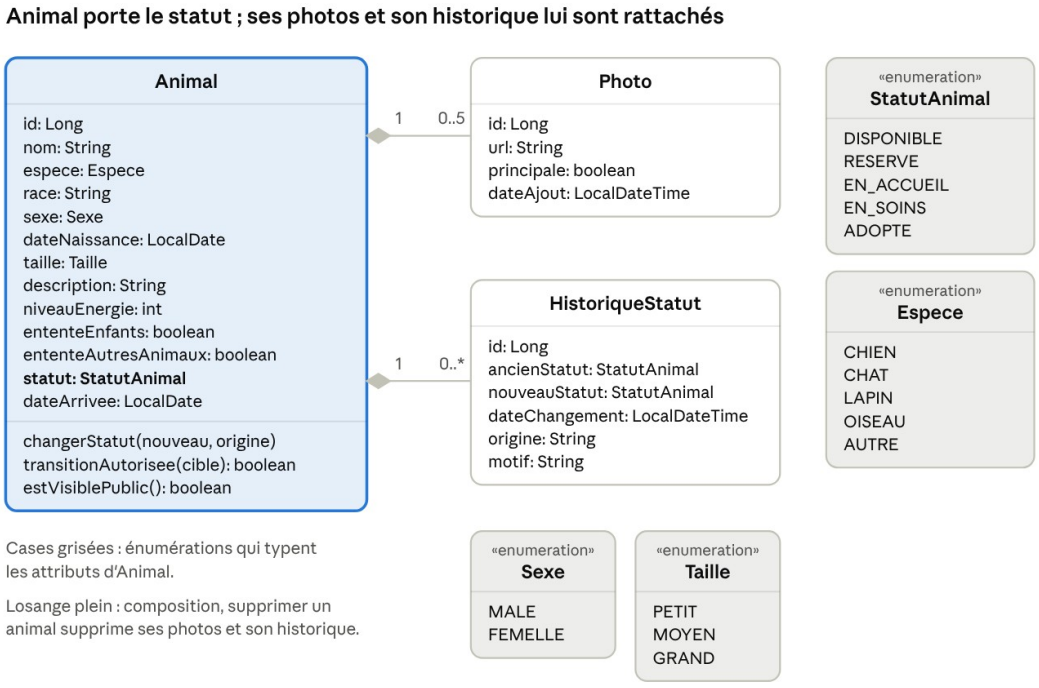


Figure 3 : MS1 · modèle de domaine (3 classes, 4 énumérations)

Communication synchrone : MS6 lit la fiche avant de confirmer une visite

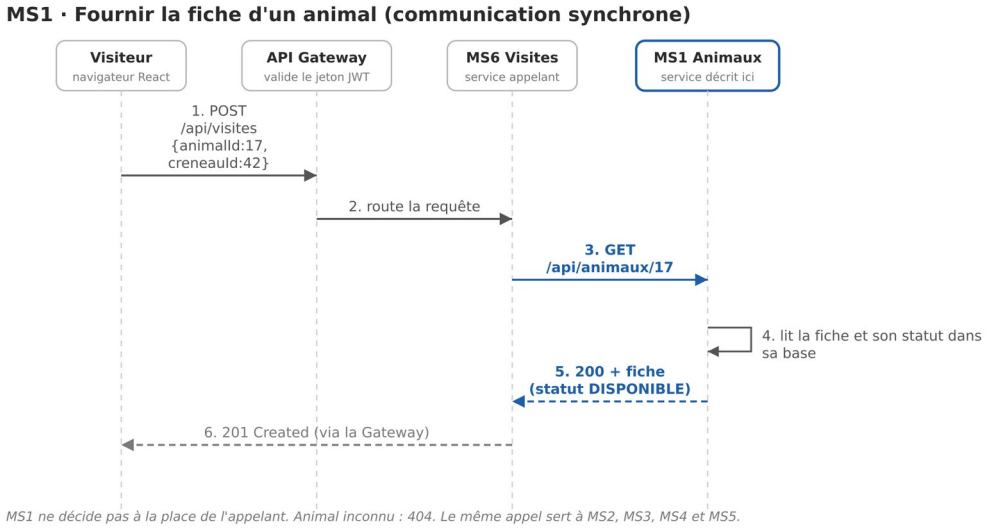


Figure 4 : MS1 · exemple de communication synchrone (appel de MS6)

Communication asynchrone : un changement de statut

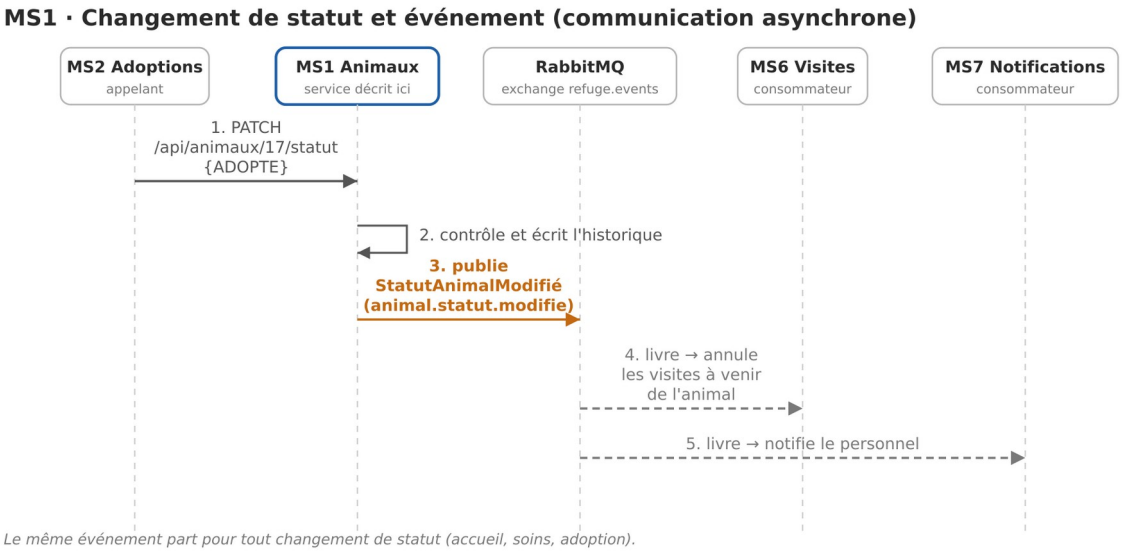


Figure 5 : MS1 · exemple de communication asynchrone (StatutAnimalModifié)

Contrat d'API

Dix endpoints sous le préfixe `/api/animaux`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/animaux?espece=&statut=&taille=&q=&page=&size=	Lister et filtrer les animaux	Tous	200, 400
GET	/api/animaux/{id}	Détail d'une fiche avec ses photos	Tous, et MS2 à MS6	200, 404
POST	/api/animaux	Créer une fiche (publie AnimalCréé)	PERSONNEL	201, 400
PUT	/api/animaux/{id}	Modifier une fiche, hors statut	PERSONNEL	200, 400, 404

Méthode	Endpoint	Description	Rôles	Codes
PATCH	/api/animaux/{id}/statut	Changer le statut (publie StatutAnimalModifié) ; appelé aussi par MS2 et MS3	PERSONNEL	200, 400, 404, 409
DELETE	/api/animaux/{id}	Supprimer une fiche saisie par erreur	ADMIN	204, 404, 409
GET	/api/animaux/{id}/historique	Parcours des statuts de l'animal	PERSONNEL	200, 404
POST	/api/animaux/{id}/photos	Ajouter une photo	PERSONNEL	201, 400, 404, 409
DELETE	/api/animaux/{id}/photos/{photoId}	Retirer une photo	PERSONNEL	204, 404
GET	/api/animaux/statistiques	Nombre d'animaux par statut et par espèce, durée moyenne de séjour	ADMIN	200

ADMIN a aussi tous les droits de PERSONNEL. Les deux premières routes sont publiques.

Événement	Clé de routage	Déclencheur	Contenu de data
AnimalCréé	animal.cree	Création d'une fiche	animalId, nom, espece, statut
StatutAnimalModifié	animal.statut.modifie	Changement de statut (accepté)	animalId, ancienStatut, nouveauStatut, origine

7.2 MS2 · Demandes d'adoption

Rôle

MS2 gère les demandes des personnes qui souhaitent adopter un animal. Il possède sa propre base MySQL et ne modifie jamais la base de MS1 : il ne stocke que `animalId`.

Diagramme de classes

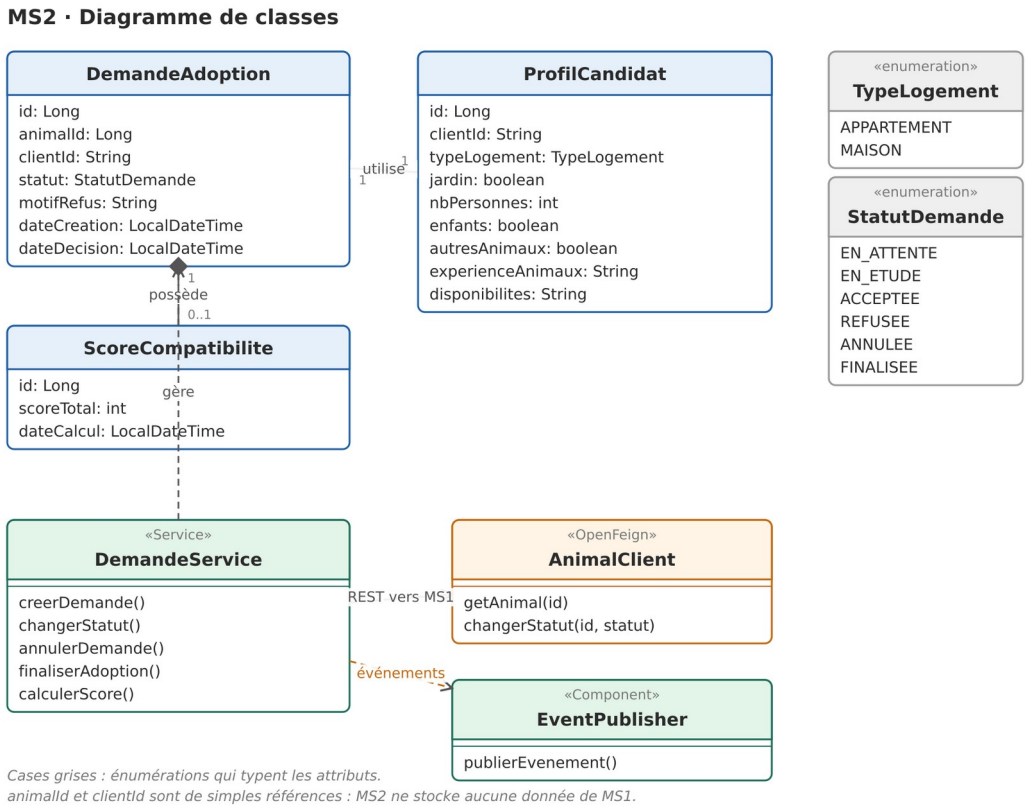


Figure 6 : MS2 · diagramme de classes

Communication synchrone : finaliser une adoption

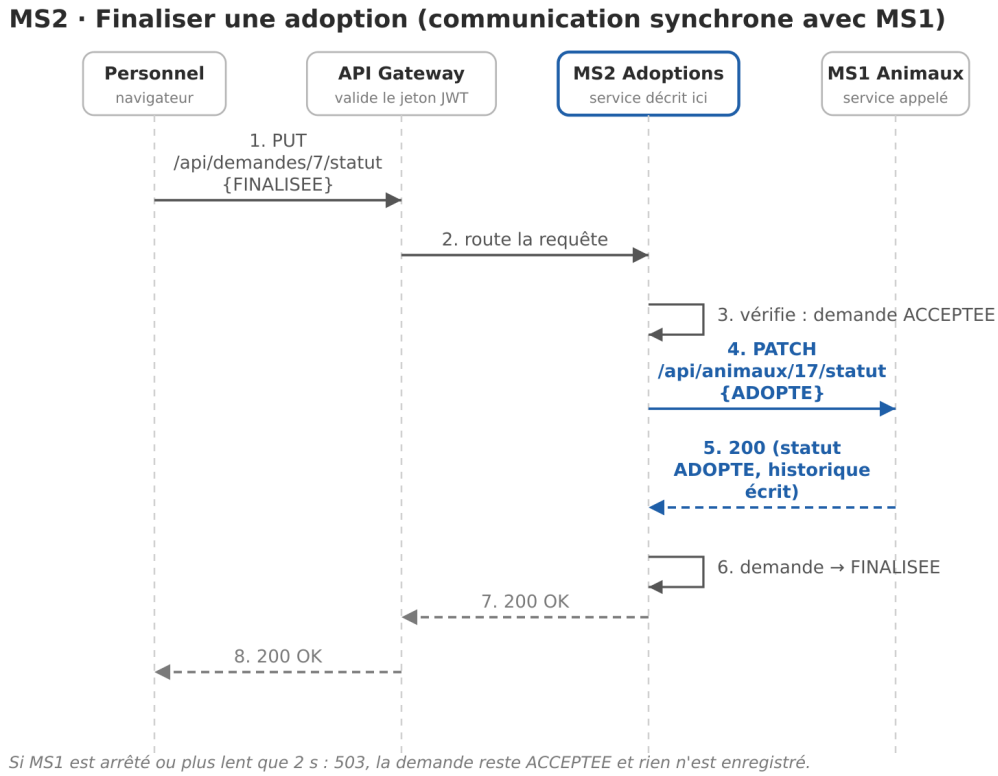


Figure 7 : MS2 · exemple de communication synchrone avec MS1

Communication asynchrone : publier AdoptionFinalisée

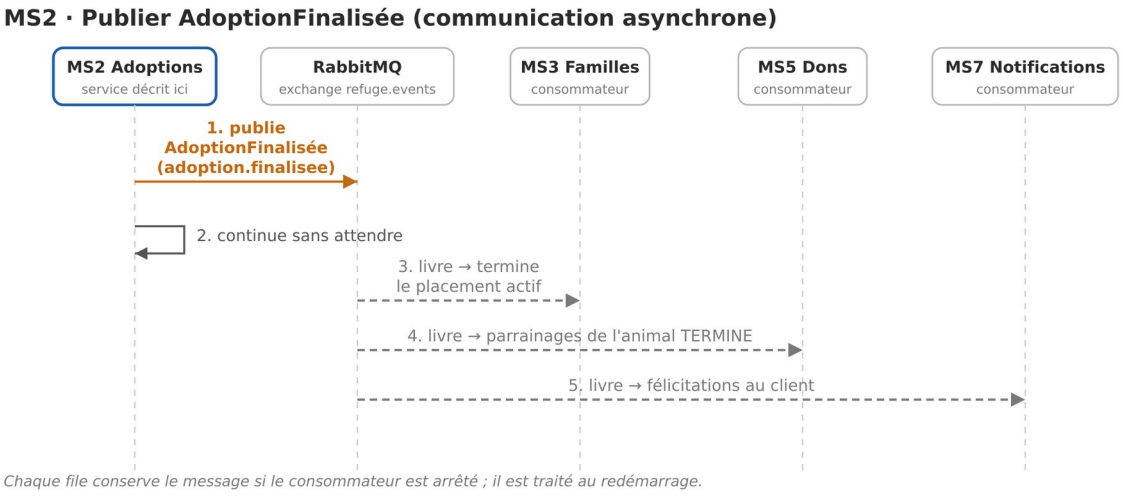


Figure 8 : MS2 · exemple de communication asynchrone

Contrat d'API

Douze endpoints. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/demandes	Créer une demande d'adoption	CLIENT	201, 400, 404, 409, 503
GET	/api/demandes	Récupérer toutes les demandes	PERSONNEL, ADMIN	200

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/demandes/{id}	Récupérer une demande par identifiant	CLIENT (la sienne), PERSONNEL, ADMIN	200, 404
GET	/api/demandes/client/{clientId}	Récupérer les demandes d'un client	CLIENT (les siennes), PERSONNEL, ADMIN	200
PUT	/api/demandes/{id}/statut	Accepter, refuser ou finaliser une demande	PERSONNEL	200, 400, 404, 409, 503
DELETE	/api/demandes/{id}	Annuler une demande	CLIENT (la sienne), PERSONNEL	200, 404, 409
POST	/api/profils	Créer un profil candidat	CLIENT	201, 400
GET	/api/profils/{id}	Consulter un profil	CLIENT (le sien), PERSONNEL	200, 404
PUT	/api/profils/{id}	Modifier un profil	CLIENT (le sien)	200, 400, 404
GET	/api/profils/client/{clientId}	Consulter le profil d'un client	CLIENT (le sien), PERSONNEL	200, 404
GET	/api/demandes/{id}/score	Calculer et consulter le score de compatibilité	PERSONNEL, ADMIN	200, 404, 503
GET	/api/demandes/{id}/compatibilite	Détails de compatibilité	PERSONNEL, ADMIN	200, 404

Appels vers MS1 : `GET /api/animaux/{id}` (existence, disponibilité et caractéristiques de l'animal) et `PATCH /api/animaux/{id}/statut` (passage à ADOPTE après adoption).

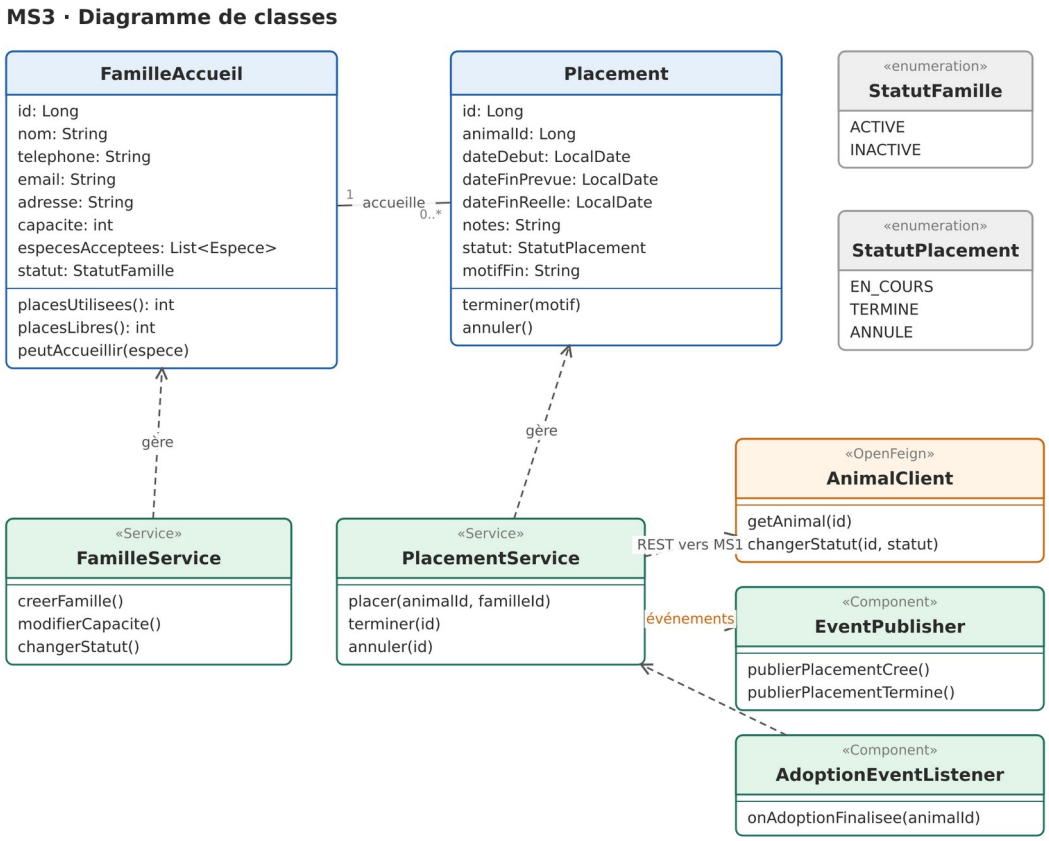
Événement	Clé de routage	Déclencheur	Contenu de data
DemandeAdoptionCréée	demande.creee	Dépôt d'une demande	demandeId, clientId, animalId
DemandeAdoptionAcceptée	demande.acceptee	Demande acceptée	demandeId, clientId, animalId
DemandeAdoptionRefusée	demande.refusee	Demande refusée	demandeId, clientId, animalId, motifRefus
AdoptionFinalisée	adoption.finalisee	Adoption finalisée (après la réponse de MS1)	demandeId, clientId, animalId

7.3 MS3 · Familles d'accueil

Rôle

MS3 gère les familles d'accueil du refuge, leur capacité et le placement temporaire d'un animal chez une famille. Il possède sa propre base MySQL et ne garde que `animalId`.

Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 (aucune copie de la fiche).

Figure 9 : MS3 · diagramme de classes

Communication synchrone : placer un animal

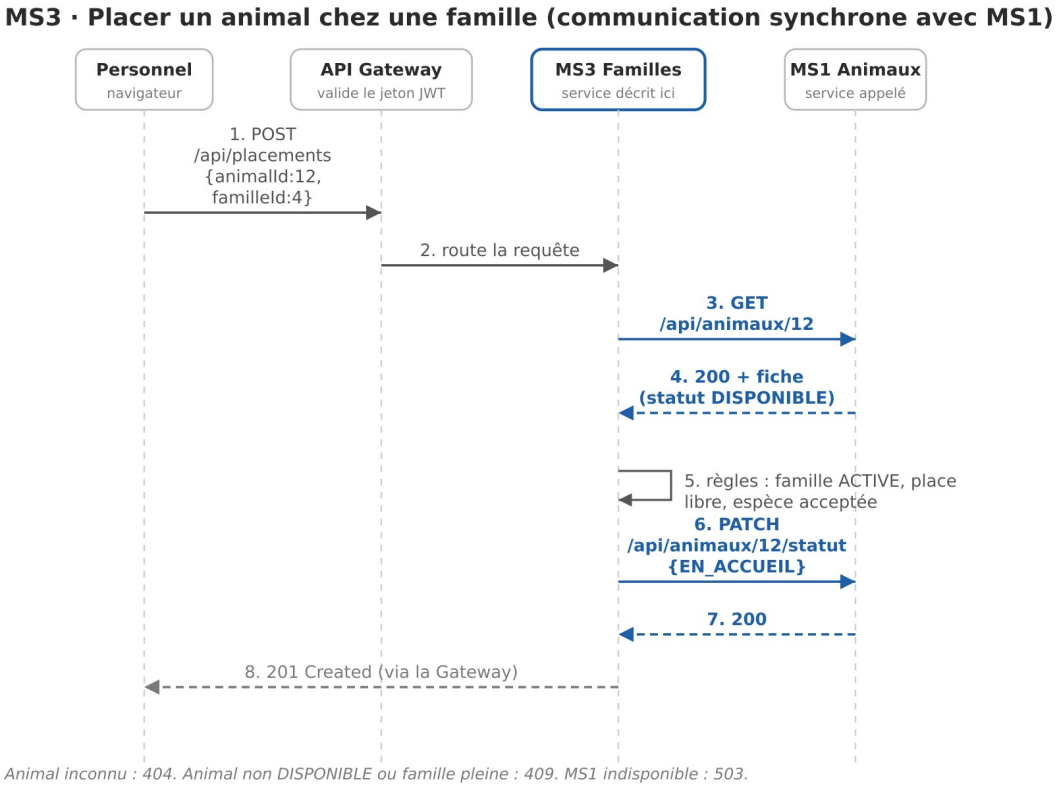


Figure 10 : MS3 · exemple de communication synchrone avec MS1

Communication asynchrone : fin d'un placement

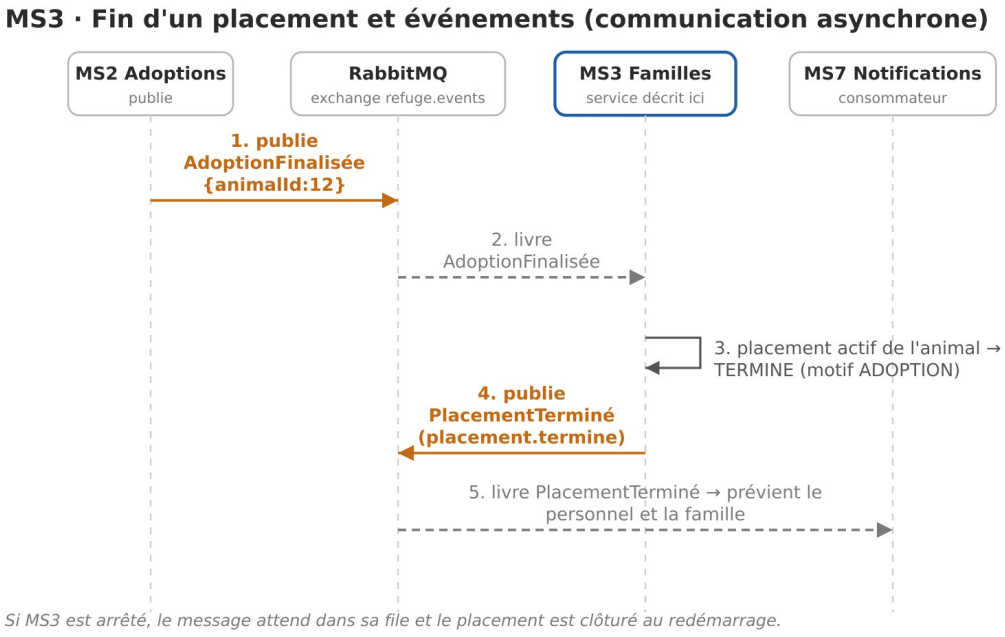


Figure 11 : MS3 · exemple de communication asynchrone

Contrat d'API

Onze endpoints, réservés à PERSONNEL et ADMIN. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/familles	Enregistrer une famille (nom, contact, adresse, capacité, espèces acceptées)	PERSONNEL	201, 400, 409
GET	/api/familles?statut=&espece=&disponibles=&page=&size=	Lister les familles	PERSONNEL	200
GET	/api/familles/{id}	Voir une famille avec ses places utilisées et libres	PERSONNEL	200, 404
PUT	/api/familles/{id}	Modifier les informations ou la capacité	PERSONNEL	200, 400, 404, 409
PATCH	/api/familles/{id}/statut	Activer ou désactiver une famille	PERSONNEL	200, 404, 409
POST	/api/placements	Placer un animal chez une famille	PERSONNEL	201, 400, 404, 409, 503
GET	/api/placements?statut=&familleId=&animalId=&page=&size=	Lister les placements	PERSONNEL	200
GET	/api/placements/{id}	Voir un placement	PERSONNEL	200, 404
PATCH	/api/placements/{id}/terminer	Terminer un placement (retour au refuge)	PERSONNEL	200, 404, 409, 503
PATCH	/api/placements/{id}/annuler	Annuler un placement fait par erreur	PERSONNEL	200, 404, 409, 503
GET	/api/familles/{id}/placements	Placements en cours et passés d'une famille	PERSONNEL	200, 404

Événement	Clé de routage	Déclencheur	Contenu de data
PlacementCréé	placement.cree	Création d'un placement	placementId, animalId, familleId, familleEmail, dateDebut, dateFinPrevue
PlacementTerminé	placement.termine	Fin ou annulation d'un placement	placementId, animalId, familleId, familleEmail, motif (RETOUR_REFUGE, ADOPTION, ANNULATION)

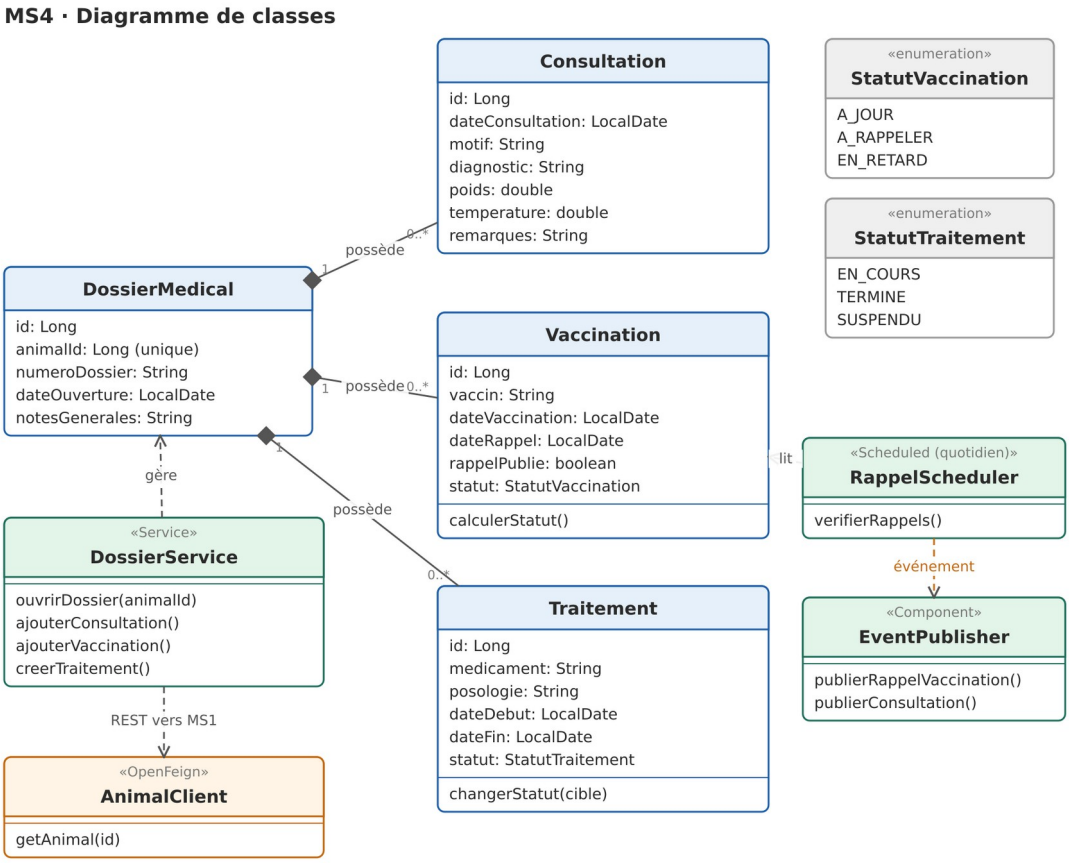
Événement reçu : `AdoptionFinalisée` (de MS2), qui termine le placement en cours de l'animal adopté.

7.4 MS4 · Suivi vétérinaire

Rôle

MS4 gère le suivi médical des animaux : dossiers médicaux, consultations, vaccinations avec rappels et traitements. Il possède sa propre base MySQL et se réfère à l'animal par `animalId` uniquement. Les visites du public et les événements restent dans MS6.

Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 (aucune table de MS1 copiée).

Figure 12 : MS4 · diagramme de classes

Communication synchrone : ouvrir un dossier

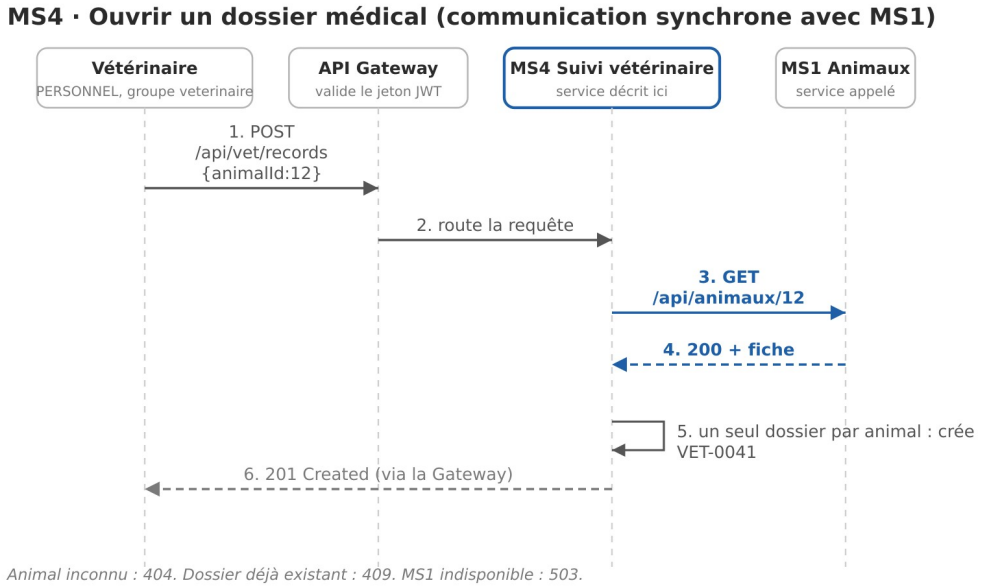


Figure 13 : MS4 · exemple de communication synchrone avec MS1

Communication asynchrone : rappel de vaccin

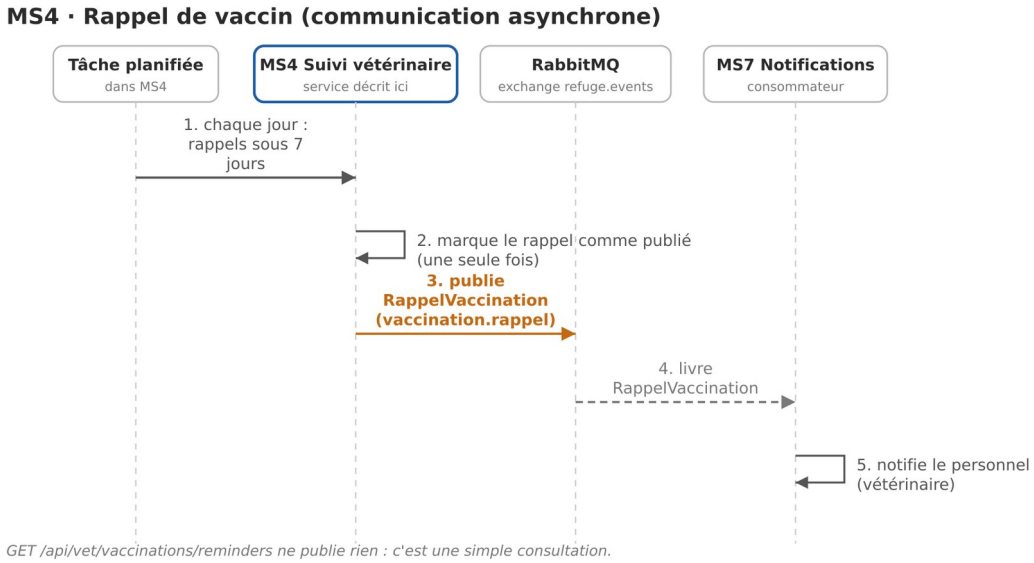


Figure 14 : MS4 · exemple de communication asynchrone

Contrat d'API

Quatorze endpoints sous le préfixe `/api/vet`. Lecture : PERSONNEL et ADMIN ; écriture : PERSONNEL du groupe « veterinaire » et ADMIN. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/vet/records	Ouvrir un dossier pour un animal (vérifie MS1)	Écriture	201, 400, 404, 409, 503
GET	/api/vet/records/{id}	Afficher un dossier	Lecture	200, 404
GET	/api/vet/records/by-animal/{animalId}	Retrouver le dossier d'un animal	Lecture	200, 404
PATCH	/api/vet/records/{id}	Modifier les notes générales	Écriture	200, 400, 404

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/vet/consultations	Enregistrer une consultation	Écriture	201, 400, 404
GET	/api/vet/records/{id}/consultations	Historique des consultations	Lecture	200, 404
GET	/api/vet/consultations/{id}	Consulter une consultation	Lecture	200, 404
POST	/api/vet/vaccinations	Enregistrer une vaccination	Écriture	201, 400, 404
GET	/api/vet/records/{id}/vaccinations	Historique des vaccinations	Lecture	200, 404
GET	/api/vet/vaccinations/reminders?beforeDate=&statut=	Rappels à venir ou en retard	Lecture	200, 400
PATCH	/api/vet/vaccinations/{id}	Rectifier une vaccination ou son rappel	Écriture	200, 400, 404
POST	/api/vet/treatments	Créer un traitement	Écriture	201, 400, 404
GET	/api/vet/records/{id}/treatments	Lister les traitements d'un dossier	Lecture	200, 404
PATCH	/api/vet/treatments/{id}/status	Changer le statut d'un traitement	Écriture	200, 400, 404, 409

Événement	Clé de routage	Déclencheur	Contenu de data
RappelVaccination	vaccination.rappel	Rappel de vaccin à venir (une seule fois par rappel)	animalId, vaccin, dateRappel
ConsultationEnregistrée	consultation.enregistrée	Nouvelle consultation	animalId, consultationId, date

7.5 MS5 · Dons et parrainages

Rôle

MS5 gère les dons faits au refuge et les parrainages mensuels d'animaux. Il possède sa propre base MySQL ; il ne gère pas les fiches animaux (MS1) et garde seulement `animalId`.

Diagramme de classes

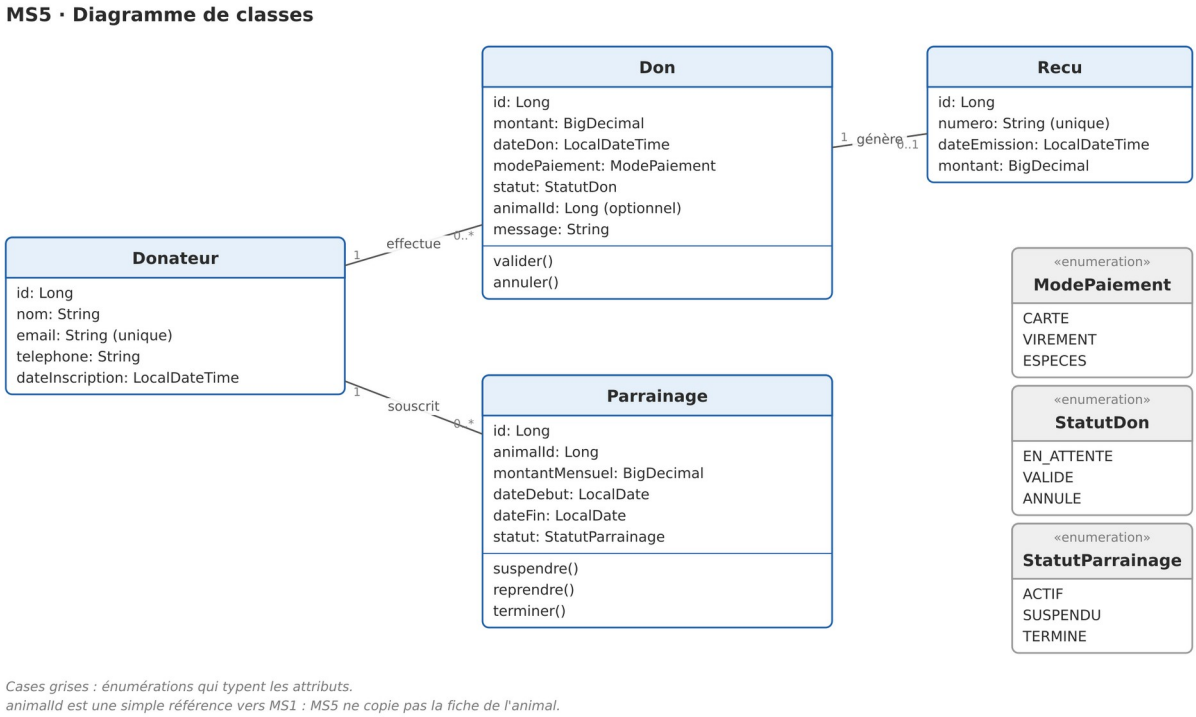


Figure 15 : MS5 · diagramme de classes

Communication synchrone : enregistrer un don pour un animal

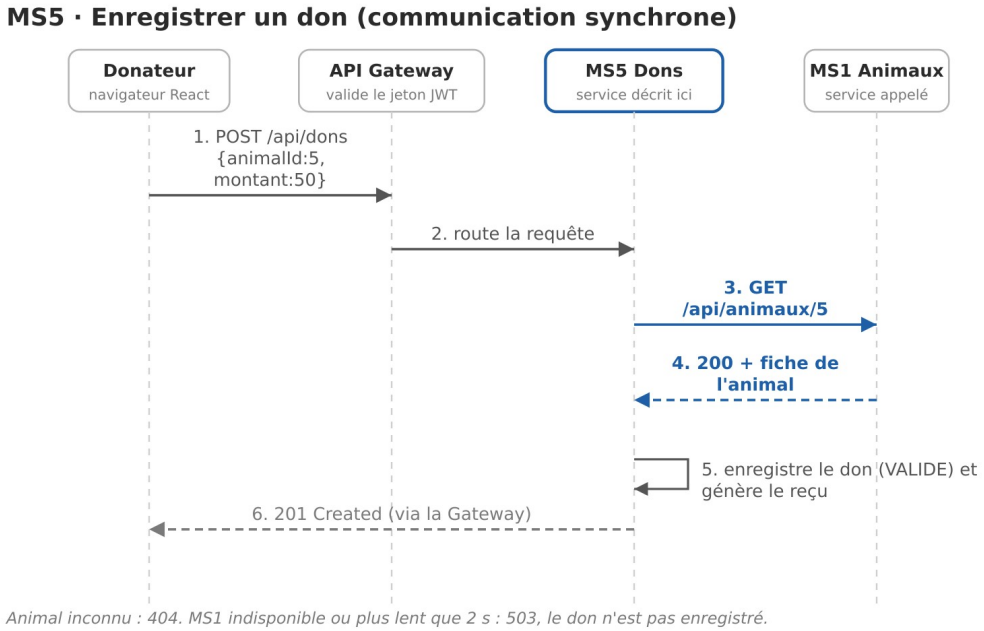


Figure 16 : MS5 · exemple de communication synchrone avec MS1

Communication asynchrone

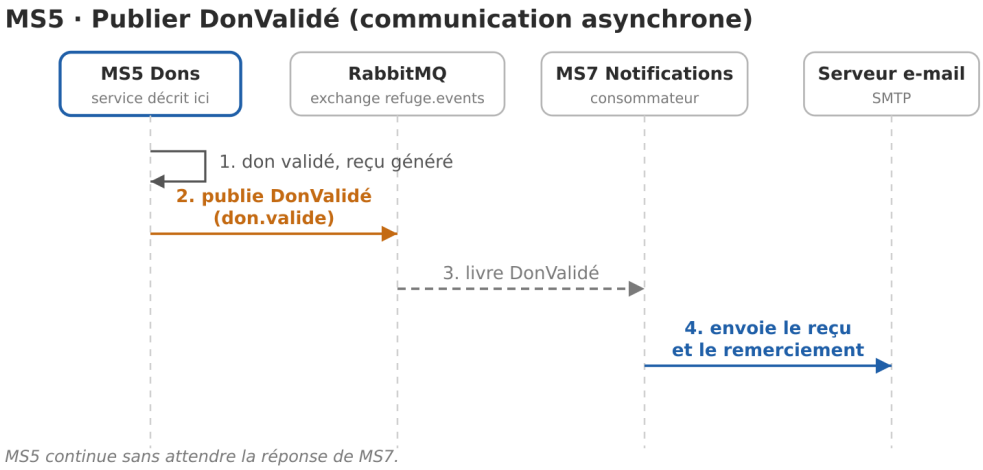


Figure 17 : MS5 · publication de DonValidé et notification du donateur

Contrat d'API

Quinze endpoints. « (propre) » signifie que le client n'accède qu'à ses propres données. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/donateurs	Créer un donateur	CLIENT, PERSONNEL, ADMIN	201, 400, 409
GET	/api/donateurs/{id}	Détail d'un donateur	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PUT	/api/donateurs/{id}	Modifier un donateur	CLIENT (propre), ADMIN	200, 400, 404
GET	/api/donateurs/{id}/dons	Historique des dons d'un donateur	CLIENT (propre), PERSONNEL, ADMIN	200, 404
POST	/api/dons	Enregistrer un don (génère un reçu, publie DonValidé)	CLIENT, ADMIN	201, 400, 404, 503
GET	/api/dons?animalId=&donateurId=	Lister et filtrer les dons	PERSONNEL, ADMIN	200, 400
GET	/api/dons/{id}	Détail d'un don	CLIENT (propre), PERSONNEL, ADMIN	200, 404
GET	/api/dons/{id}/recu	Obtenir le reçu d'un don	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PATCH	/api/dons/{id}/annuler	Annuler un don	ADMIN	200, 404, 409
POST	/api/parrainages	Créer un parrainage (publie ParrainageCréé)	CLIENT	201, 400, 404, 409, 503
GET	/api/parrainages?animalId=&statut=	Lister et filtrer les parrainages	PERSONNEL, ADMIN	200, 400
GET	/api/parrainages/{id}	Détail d'un parrainage	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PATCH	/api/parrainages/{id}/statut	Suspendre, reprendre ou terminer (publie ParrainageTerminé si terminé)	CLIENT (propre), ADMIN	200, 400, 404, 409
GET	/api/animaux/{animalId}/parrains	Lister les parrains d'un animal	PERSONNEL, ADMIN	200, 404
GET	/api/stats/dons?debut=&fin=	Total collecté sur une période et animaux les plus soutenus	PERSONNEL, ADMIN	200, 400

Événement	Clé de routage	Déclencheur	Contenu de data
DonValidé	don.valide	Don validé et reçu généré	donId, donateurId, donateurEmail, animalId (optionnel), montant, numeroRecu
ParrainageCréé	parrainage.cree	Création d'un parrainage	parrainageId, donateurId, donateurEmail, animalId, montantMensuel
ParrainageTerminé	parrainage.termine	Fin d'un parrainage (manuelle ou automatique)	parrainageId, donateurId, donateurEmail, animalId, motif

Événement reçu : `AdoptionFinalisée` (de MS2), qui contient au minimum `animalId`.

7.6 MS6 · Visites et rendez-vous

Rôle

MS6 gère les **visites** des animaux au refuge (créneaux et réservations) et les **événements** (journées d'adoption, ateliers) avec leurs inscriptions et leur liste d'attente. Un refuge doit faire venir du monde pour trouver des adoptants, des donateurs et des bénévoles. Une visite est individuelle (une personne voit un animal sur un créneau) ; un événement est collectif (date, lieu, places limitées). Les deux reposent sur la même logique de places limitées.

Diagramme de classes

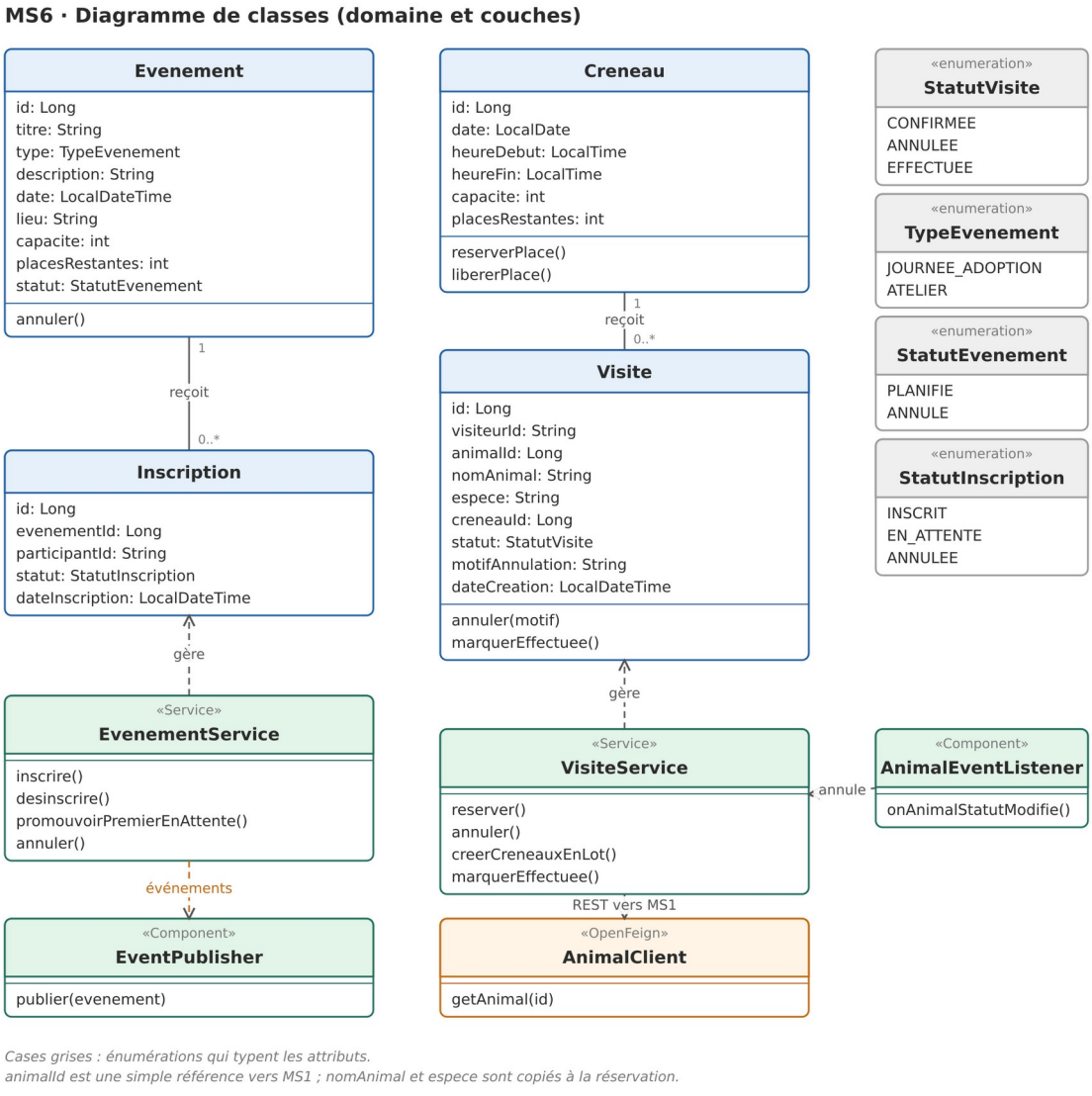


Figure 18 : MS6 · diagramme de classes (domaine et couches)

Communication synchrone : réserver une visite

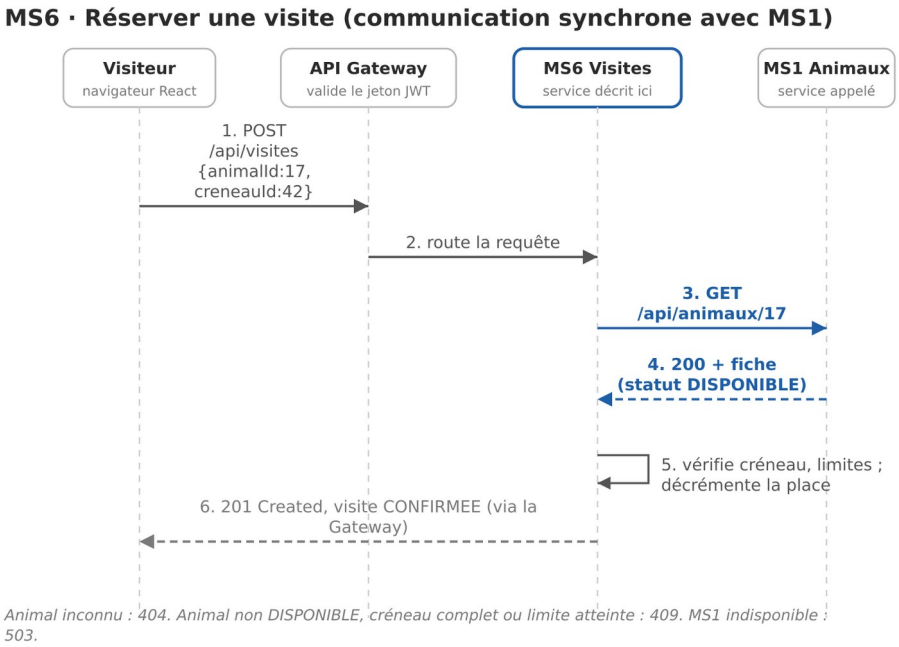


Figure 19 : MS6 · exemple de communication synchrone avec MS1

Communication asynchrone

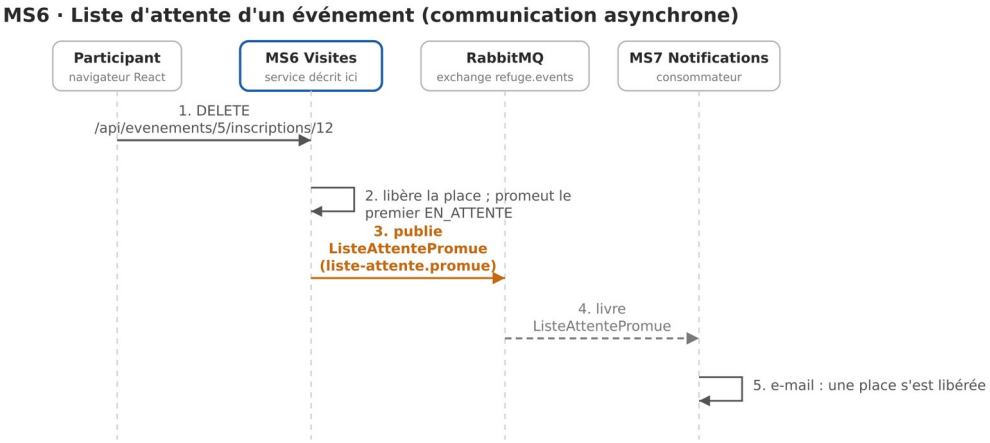


Figure 20 : MS6 · promotion de la liste d'attente

Contrat d'API

Quinze endpoints sous les préfixes `/api/visites` et `/api/evenements`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/visites/creneaux	Créer un créneau	PERSONNEL	201, 400
POST	/api/visites/creneaux/lot	Créer une série de créneaux	PERSONNEL	201, 400
GET	/api/visites/creneaux?date=	Lister les créneaux disponibles	Tous	200
PUT	/api/visites/creneaux/{id}	Modifier un créneau (annule les visites si date ou heure change)	PERSONNEL	200, 400, 404, 409
DELETE	/api/visites/creneaux/{id}	Supprimer un créneau (annule ses visites)	PERSONNEL	204, 404

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/visites	Réserver une visite	CLIENT, PERSONNEL	201, 400, 404, 409, 503
GET	/api/visites?visiteurId=&animalId=	Lister les visites	CLIENT (les siennes), PERSONNEL	200
DELETE	/api/visites/{id}	Annuler une visite (24 h avant maximum pour un visiteur)	CLIENT (la sienne), PERSONNEL	204, 403, 404, 409
PATCH	/api/visites/{id}/effectuee	Marquer une visite comme effectuée	PERSONNEL	200, 404, 409
POST	/api/evenements	Créer un événement	PERSONNEL	201, 400
GET	/api/evenements?aVenir=true	Lister les événements	Tous	200
PATCH	/api/evenements/{id}/annuler	Annuler un événement	ADMIN	200, 404, 409
POST	/api/evenements/{id}/inscriptions	S'inscrire (ou entrer en liste d'attente)	Tous (connectés)	201, 404, 409
DELETE	/api/evenements/{id}/inscriptions/{inscriptionId}	Se désinscrire (promeut le premier en attente)	CLIENT (la sienne), PERSONNEL	204, 403, 404
GET	/api/evenements/{id}/inscriptions	Lister les inscrits et la liste d'attente	PERSONNEL	200, 404

Événement	Clé de routage	Déclencheur	Contenu de data
VisiteCréée	visite.creee	Réservation d'une visite	visiteId, visiteurId, animalId, date
VisiteAnnulée	visite.annulee	Annulation d'une visite	visiteId, visiteurId, animalId, motif
EvenementAnnulé	evenement.annule	Annulation d'un événement	evenementId, titre, date, participantIds
ListeAttentePromue	liste-attente.promue	Promotion du premier en attente	evenementId, titre, participantId

Événement reçu : `StatutAnimalModifié` (clé `animal.statut.modifie`, de MS1) : `{ animalId, ancienStatut, nouveauStatut }`. Si le nouveau statut n'est pas DISPONIBLE, MS6 annule les visites à venir de l'animal.

7.7 MS7 · Notifications (service commun)

Rôle

MS7 est le service commun, développé par toute l'équipe. Il écoute les événements publiés par les autres services et en fait des notifications : e-mails (reçu de don, confirmation de visite, décision sur une demande d'adoption, etc.) et notifications internes pour le personnel. Il utilise **MongoDB**, la technologie avancée du projet : chaque notification est un document, et les contenus varient beaucoup d'un événement à l'autre.

Notifications produites

Événement reçu	Destinataire	Notification
AnimalCréé	PERSONNEL	Nouvel animal enregistré (interne)
StatutAnimalModifié	PERSONNEL	Changement de statut d'un animal (interne)
DemandeAdoptionCréée / Acceptée / Refusée	Client	Accusé de réception, acceptation, refus avec motif (e-mail)
AdoptionFinalisée	Client	Félicitations pour l'adoption (e-mail)
PlacementCréé / PlacementTerminé	Famille, PERSONNEL	Début et fin d'un placement (e-mail à la famille, interne au personnel)
RappelVaccination	PERSONNEL	Vaccin à renouveler avant la date de rappel (interne)
ConsultationEnregistrée	PERSONNEL	Nouvelle consultation (interne)
DonValidé	Donateur	Reçu et remerciement (e-mail)
ParrainageCréé / ParrainageTerminé	Parrain	Confirmation et fin du parrainage (e-mail)
VisiteCréée / VisiteAnnulée	Visiteur	Confirmation et annulation de visite (e-mail)
EvenementAnnulé / ListeAttentePromue	Participants / participant	Annulation d'un événement ; place obtenue en liste d'attente (e-mail)

Diagramme de classes

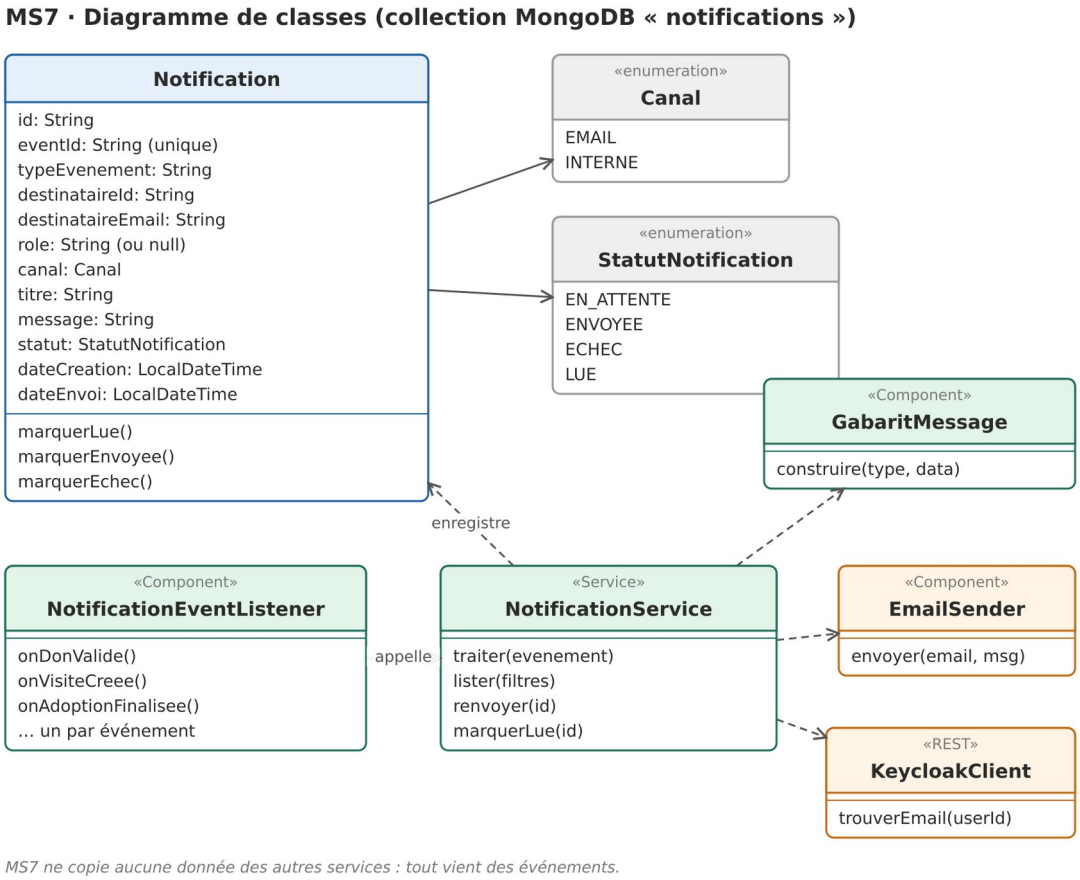


Figure 21 : MS7 · diagramme de classes

Communication asynchrone : de l'événement à l'e-mail

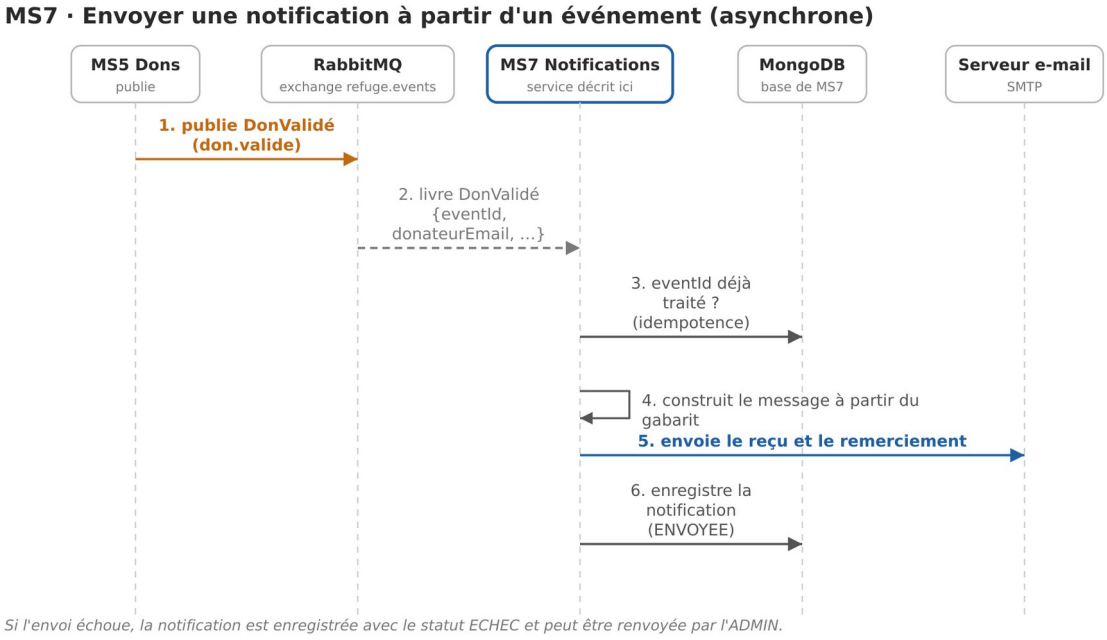


Figure 22 : MS7 · exemple de communication asynchrone

Communication synchrone : consulter ses notifications

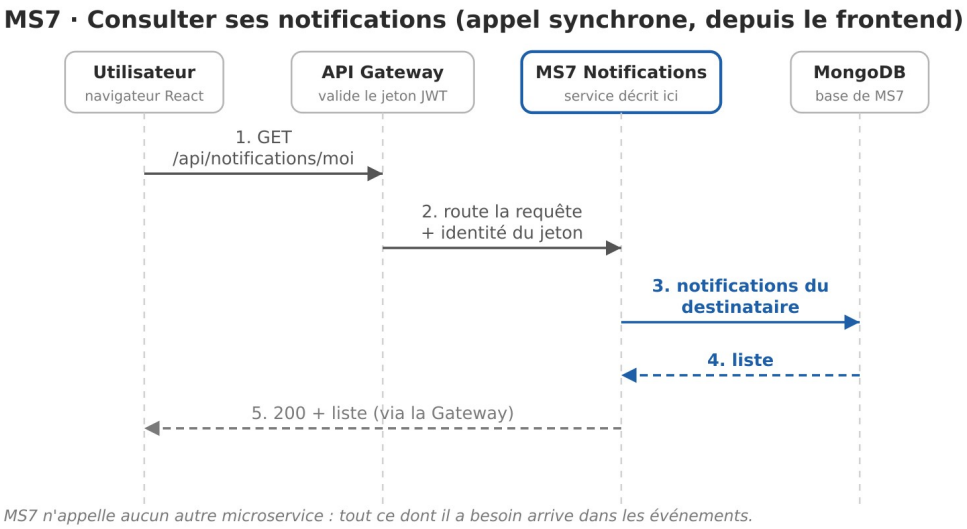


Figure 23 : MS7 · exemple de communication synchrone (appel du frontend)

Contrat d'API

Cinq endpoints sous le préfixe `/api/notifications`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/notifications/moi?statut=&page=&size=	Mes notifications	CLIENT, PERSONNEL, ADMIN	200
GET	/api/notifications/{id}	Détail d'une notification	Son destinataire, PERSONNEL, ADMIN	200, 403, 404
PATCH	/api/notifications/{id}/lue	Marquer comme lue	Son destinataire	200, 403, 404
GET	/api/notifications?type=&statut=&page=&size=	Lister toutes les notifications	ADMIN	200, 400
POST	/api/notifications/{id}/renvoyer	Renvoyer une notification en échec	ADMIN	200, 404, 409

Événements reçus : tous ceux de la section 6.2. MS7 ne publie aucun événement.

8. Technologies et conventions communes

8.1 Technologies choisies

Besoin	Technologie	Justification
Microservices	Java 17, Spring Boot 3.5, Spring Data JPA, Maven	Stack imposée ; même structure que l'exemple du cours
API Gateway, découverte	Spring Cloud Gateway, Eureka	Stack imposée ; point d'entrée unique, services trouvés par leur nom
Sécurité	Keycloak (OIDC, JWT)	Stack imposée ; comptes, rôles et groupes
Bases de données	MySQL (MS1 à MS6), MongoDB (MS7), H2 pour les tests	Une base par service ; MongoDB pour le service commun (technologie avancée)
Synchrone	OpenFeign, Resilience4j	Appels REST entre services avec délai maximal et coupe-circuit (503)
Asynchrone	RabbitMQ (Spring AMQP)	Événements découplés, files durables, routage par clé
Documentation, déploiement	Swagger/OpenAPI, Docker, Docker Compose	Stack imposée ; un fichier démarre toute l'application
Frontend, suivi	React ; Git, GitHub, GitHub Projects	Application web ; dépôt public et tableau de tâches

8.2 Conventions communes

- Préfixe `/api`, format JSON, dates ISO 8601 ; documentation Swagger sur chaque service ; les identifiants d'utilisateur sont lus dans le jeton Keycloak, jamais dans le corps d'une requête.
- Erreurs : même corps JSON (`timestamp, status, error, message, path`) produit par un `GlobalExceptionHandler` dans chaque service.

Code	Signification
200 / 201 / 204	Succès, création, succès sans contenu
400	Données invalides
401 / 403	Non authentifié / rôle insuffisant ou ressource d'un autre utilisateur
404	Ressource introuvable (y compris un animal inconnu de MS1)
409	Conflit avec une règle métier (doublon, capacité atteinte, transition interdite)
503	Service appelé indisponible (MS1 arrêté ou plus lent que 2 s)

Structure de chaque microservice (celle de l'exemple vu en cours, `condidat_s1`) :

```
msX-nom/
├── database/                        données de test (SQL)
├── src/main/java/com/awd/<service>/
│   ├── controller/  service/  repository/  entity/  dto/  mapper/
│   └── common/{config, exception}/
├── src/main/resources/application.properties
├── src/test/java/                   tests d'intégration (H2)
└── pom.xml, README.md
```

9. Répartition des tâches

Service	Responsable	À réaliser
MS1 Animaux	Youssef	Fiches, statuts, photos, historique, statistiques ; événements AnimalCréé et StatutAnimalModifié
MS2 Demandes d'adoption	Zeineb	Profils, demandes, score, finalisation ; appels vers MS1 ; 4 événements
MS3 Familles d'accueil	Aziz	Familles, capacité, placements ; appels vers MS1 ; événements de placement ; écoute d'AdoptionFinalisée
MS4 Suivi vétérinaire	Emna	Dossiers, consultations, vaccinations, traitements ; rappels planifiés ; 2 événements
MS5 Dons et parrainages	Lazhar	Donateurs, dons, reçus, parrainages, statistiques ; 3 événements ; écoute d'AdoptionFinalisée
MS6 Visites et rendez-vous	Kmar	Créneaux, visites, événements, inscriptions, liste d'attente ; 4 événements ; écoute de StatutAnimalModifié
MS7 Notifications (commun)	Toute l'équipe	Squelette, puis un écouteur et un message par membre pour les événements de son service

Tâches communes : API Gateway et routes, Eureka, Keycloak (royaume, rôles, groupe « veterinaire »), RabbitMQ et Docker Compose, frontend React (chaque membre réalise les écrans de son service), dossier technique et présentation (parties réparties équitablement ; chacun présente son microservice en 1 minute ; un membre présente MS7). Un responsable est désigné pour chaque tâche commune dans GitHub Projects.

10. Organisation du dépôt Git et gestion de projet

10.1 Dépôt

Dépôt public `refugia-4DAS_I13-InSync` sur GitHub ; l'enseignant et les membres de l'équipe sont collaborateurs. Il contient `Backend/` (un dossier par microservice), `Frontend/` et `Documentation/` (`Presentation/`, `Diagrams/`, ce dossier technique), plus le `README.md`.

- `main` contient la version stable ; une branche par fonctionnalité (`feature/ms6-creneaux`) ; fusion par Pull Request relue par un autre membre.
- Chaque membre travaille dans le dossier de son service, ce qui limite les conflits ; messages de commit courts préfixés par le service.

10.2 Outil de gestion de projet

GitHub Projects, un tableau commun lié au dépôt, avec les colonnes **À faire**, **En cours**, **En revue**, **Terminé**. Chaque tâche est une carte rattachée à un service et assignée à un membre, ce qui montre aussi la répartition du travail.

11. Planning prévisionnel et risques

11.1 Planning

Séance	Objectifs	Livrables
S1 à S3	Équipe, sujet, découpage en microservices, contrats d'API	Idée, découpage, premiers contrats
S4 · Jalon 1	Valider la conception	Dépôt Git, dossier technique, présentation, diagramme d'architecture

Séance	Objectifs	Livrables
S5	Gateway, Eureka, CRUD de chaque service, Swagger	Services joignables par la Gateway ; MS1 en priorité
S6	Communication synchrone (OpenFeign, délai, coupe-circuit)	Appels vers MS1 dans chaque service
S7	Communication asynchrone (RabbitMQ), MS7	17 événements, notifications de bout en bout
S8	Sécurité Keycloak : rôles, JWT, droits par endpoint	Connexion, trois rôles, groupe « veterinaire »
S9	Docker, Docker Compose, frontend, tests	Démarrage de toute l'application
S10 · Jalon 2	Démonstration et soutenance	Application complète, code de chaque service

11.2 Principaux risques

Risque	Réponse prévue
Contrats d'API ou noms qui divergent entre services	Conventions communes (section 8) et catalogue d'événements (section 6.2) validés ensemble ; Swagger pour vérifier
MS1 en retard : tous les services en dépendent	MS1 en priorité à la S5 ; données de test partagées ; réponse 503 prévue côté appelant
Événements perdus ou traités deux fois	Files durables, `eventId` unique, idempotence chez le consommateur
Sécurité Keycloak (S8) et Docker (S9) tardifs	Règles de rôle écrites dès maintenant dans les contrats ; Dockerfile par service dès qu'il démarre
Retard ou absence d'un membre, conflits Git	Suivi GitHub Projects, point d'équipe hebdomadaire, un dossier par service, Pull Requests relues
Service commun MS7 laissé à la fin	Squelette posé à la S5 ; chaque membre ajoute son écouteur avec ses événements